RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair

André Silva, Sen Fang, and Martin Monperrus

***Abstract*—Automated Program Repair (APR) has evolved significantly with the advent of Large Language Models (LLMs). Fine-tuning LLMs for program repair is a recent avenue of research, with many dimensions which have not been explored. Existing work mostly fine-tune LLMs with naive code representations and does not scale to frontier models. To address this problem, we propose RepairLLaMA, a novel program repair approach that 1) identifies optimal code representations for APR with fine-tuned models, and 2) pioneers state-of-the-art parameter-efficient fine-tuning technique (PEFT) for program repair. This results in RepairLLaMA producing a highly effective 'program repair adapter' for fixing bugs with AI. Our experiments demonstrate the validity of both concepts. First, fine-tuning adapters with program repair specific code representations enables the model to use meaningful repair signals and produce better patches. Second, parameter-efficient fine-tuning helps fine-tuning to converge and clearly contributes to the effectiveness of RepairLLaMA in fixing bugs outside the fine-tuning data distribution. Overall, RepairLLaMA correctly fixes 144 Defects4J v2, 109 HumanEval-Java, and 20 GitBug-Java bugs, outperforming all baselines.**

***Index Terms*—Automated program repair, large language models, code representations, parameter-efficient fine-tuning.**

I. Introduction

AUTOMATED program repair (APR) [1], [2] aims at automatically fixing a software bug without human intervention. Learning-based repair [3], [4], [5], [6], [7], [8], [9], [10] has become the mainstream solution to this problem due to the powerful ability of deep neural networks to learn complex bug fix patterns. Large language models (LLMs), pre-trained on vast amounts of data, have pushed learning-based repair to the next frontier [8], [11]. There are two main research lines in applying LLMs to program repair: (1) fine-tuning techniques to adapt pre-trained LLMs and create specialized repair models [5], [8], [10], [12], and (2) prompt engineering and agent-based approaches that leverage LLMs' capabilities through carefully designed inputs and interaction patterns [13], [14], [15], [16], [17].

Fine-tuning has the potential to learn domain-specific representations and leverage signals in order to generate high-quality patches. The drawback is that it requires substantial computational resources and high-quality training data. Prompt engineering and agent-based approaches mostly consist of ad hoc human-developed prompts and workflows to instruct models for specific tasks and are bounded by the limited size of context windows. While both approaches offer different trade-offs, we argue that fine-tuning is crucial for high-quality automated program repair because: (1) it allows the model to internalize repair patterns from a supervised corpus of bug fixes, (2) it enables learning of task-specific repair representations rather than relying on manual prompt engineering steps, and (3) it provides opportunities to incorporate repair-specific information during training. Our paper is a key contribution in the space of fine-tuning LLMs for program repair.

Fine-tuning LLMs for program repair is complex. Early work simply refines the network weights based on additional fine-tuning data. However, this kind of fine-tuning is rather primitive and suffers from two significant drawbacks. First, fine-tuning is also known to be able to adapt the input/output representations of the data under study [18]. In the context of program repair, there is an opportunity to fine-tune with code representations that maximize downstream task performance, that is, repair performance. In particular, previous work overlooks the realistic representation of fault localization in the input. Second, previous work considered the most basic fine-tuning technique, which is full-parameter fine-tuning. As LLMs increase in size [19], full-parameter fine-tuning poses important overfitting problems when fine-tuning data is limited, which is typically the case in program repair. In this paper, we address the problem of devising efficient fine-tuning techniques [20] for program repair, with a focus on code representations and adapters.

We propose RepairLLaMA, a new program repair approach that leverages parameter-efficient fine-tuning to adapt LLMs to handle repair-specific code representations. First, RepairLLaMA's code representations incorporate fault localization signals and are designed to support multi-location bugs. Second, RepairLLaMA utilizes Low-Rank Adaption (LoRA), a state-of-the-art parameter-efficient fine-tuning technique, to train a much smaller *repair adapter* (when compared to the full LLM) that adapts the LLM for program repair while helping prevent

Received 7 June 2024; revised 30 May 2025; accepted 2 June 2025. Date of publication 18 June 2025; date of current version 15 August 2025. This work was supported in part by Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. Recommended for acceptance by K. Tantithamthavorn. *(André Silva and Sen Fang contributed equally to this work.) (Corresponding author: André Silva.)*

André Silva and Martin Monperrus are with KTH Royal Institute of Technology, 114 28 Stockholm, Sweden (e-mail: andreans@kth.se; monperrus@kth.se).

Sen Fang is with NC State University, Raleigh, NC 27695 USA (e-mail: sfang9@ncsu.edu).

Digital Object Identifier 10.1109/TSE.2025.3581062

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

overfitting [21]. As we will demonstrate in this paper, the concept of *repair adapter* is novel and potent.

Our experimental results validate RepairLLaMA's core design. First, RepairLLaMA achieves state-of-the-art fine-tuning performance in three benchmarks, correctly fixing 144 Defects4J v2 [22] bugs, and 109 and 20 bugs, respectively, on recently proposed benchmarks HumanEval-Java [8] and GitBug-Java [23], which boosts internal and external validity. The experiments show that the devised code representations with repair signals allow the LLM to synthesize patches more effectively than the naive code-only representations. Also, RepairLLaMA clearly outperforms non-fine-tuned baselines, incl. GPT-4. Moreover, our results also show the effectiveness of parameter-efficient fine-tuning: RepairLLaMA's repair adapters, with only 4M parameters, are 1600x smaller than the initial pre-trained LLM, CodeLLama-7B [24]. To sum up, the efficient representations and repair adapters of RepairLLaMA outperform recent results on fine-tuning for program repair [8], [10], [25] as well as world-class models such as GPT-3.5 and GPT-4.

Overall, we make the following contributions:

- We design RepairLLaMA, an original fine-tuning pipeline for automated program repair with LLMs. RepairLLaMA's representations maximize knowledge from the program repair domain, while keeping strong alignment with pre-training.
- We systematically evaluate different code representations for program repair fine-tuning. Our results clearly show that the best code representation leverages the task-specific signals of fault localization and original buggy code.
- We demonstrate that parameter-efficient fine-tuning performs competitively with full-parameter fine-tuning in the context of program repair. The "repair adapters" of RepairLLaMA are training-efficient, and achieve state-of-the-art repair performance across three benchmarks, Defects4J, HumanEval-Java, and GitBug-Java, outperforming even GPT-4.
- For the sake of open science, we publish our source code, models, and artifacts at https://github.com/ASSERT-KTH/repairllama.

II. REPAIRLLAMA: EFFICIENT FINE-TUNING FOR PROGRAM REPAIR

A. Overview

Fig. 1 illustrates the pipeline of RepairLLaMA for APR, which is divided into three consecutive stages. The core novelties of this pipeline are: 1) the APR specific code representations, and 2) the end-to-end use of a parameter-efficient fine-tuning technique.

The core of RepairLLaMA is a *repair adapter*. A *repair adapter* is a plug-and-play extension of the model parameters that modifies the behavior of the LLM in order to maximize performance on the repair task, for a given programming language. The adapter is responsible for transforming a rich, tailored input representation of the buggy code into the fit output representation of the patch.

In the first stage of RepairLLaMA, the core choices are made, namely: 1) the initial pre-trained model (subsection II-C); 2) the input code representation and output code representation (subsection II-D); and 3) the fine-tuning dataset (subsection II-E). These choices are all important and are further discussed in the remainder of this section.

In the second stage, a repair adapter is trained. The repair adapter is a much smaller (i.e., approx. 4M parameters) plug-and-play adapter of the initial LLM while remaining competitive on the task of program repair.

Finally, in the third stage, the repair adapter is employed to fix real-world bugs.

B. Target Bugs

The first consideration when designing a fine-tuning pipeline for program repair is the bugs we aim to fix. This relates to 1) the programming language, 2) the type of bugs (syntax errors, runtime errors, functional errors, etc), and 3) the difficulty of bugs, which can be proxied by the code span to modify in order to fix the bug.

In this work, we focus on 1) Java bugs, 2) that are functional, and come with at least a failing test case, 3) that are intra-procedural, i.e. fixed with changes to a single function (called hereafter *single-function bugs*). We do not make any assumption on the length of the method under repair, and 4) explicitly support bugs that require changes in multiple locations in the function [26], beyond single-line or single-chunk bugs.

C. Choice of the Initial LLM

Choosing the suitable initial LLM for fine-tuning is crucial. For example, when fine-tuning for code-related tasks, an LLM pre-trained on large-scale code corpora is more effective than one pre-trained on pure natural language data. To effectively fine-tune an LLM for APR, we curate three criteria to choose the initial model.

First, the LLM should be publicly available and open-source. Fine-tuning a closed-source LLM on the task-specific dataset is not a valid option. Although some companies like OpenAI do provide an API for fine-tuning their LLMs, it is expensive, and the ownership of the final model (incl. weights) does not meet open-science reproduction criteria. Open-source models, such as LLaMA [27] or StarCoder [28], publish model weights online, allowing anyone to modify and deploy them.

Second, the LLM should be pre-trained with large-scale code data. As observed by related work [24], [28], LLMs pre-trained on massive code data achieve better performance in code-related tasks. Thus, we consider only LLMs specialized on code.

Third, the initial LLM should have been trained with an infilling objective [29] during pre-training. As observed by related work [8], infilling is a natural and effective learning objective for the program repair task, since it allows the model to synthesize code according to both the context appearing before and after. It should also be supported by an off-the-shelf parameter-efficient fine-tuning library.

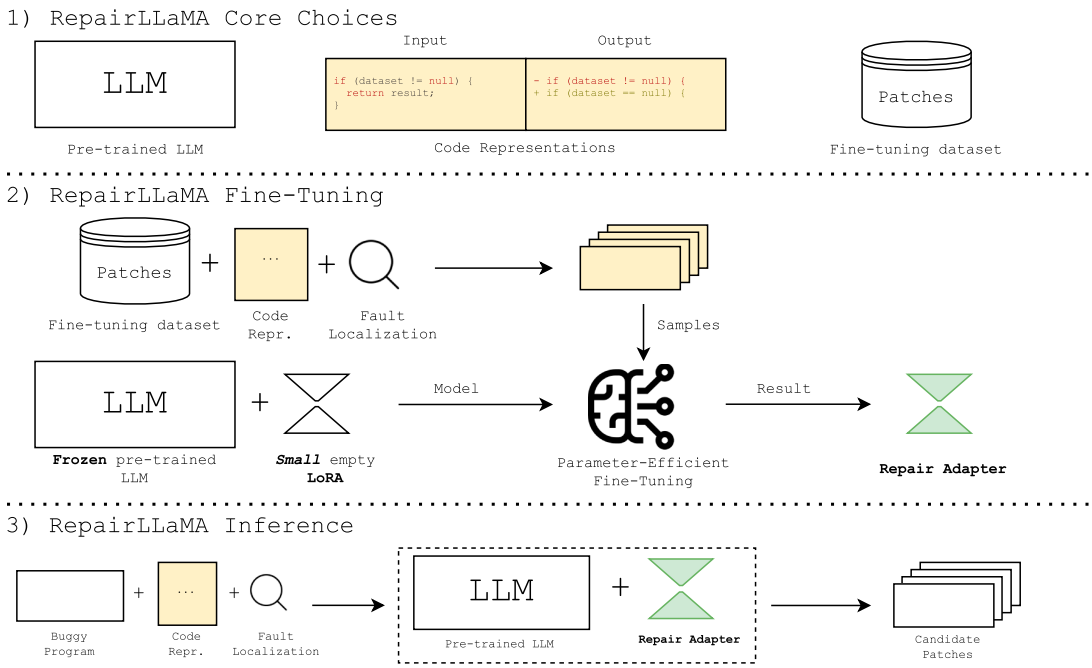


Fig. 1. Overview of RepairLLaMA. The core novelties of RepairLLaMA are the APR specific code representations and the engineering of an effective *program repair adapter* that is plugged into the underlying LLM.

In subsection III-B we instantiate those criteria in the context of functional program repair for Java.

D. Choice of Code Representations

Source code representation is a critical aspect that significantly impacts the effectiveness of the model [30]. In this section, we discuss key characteristics of the source code representation design space. We introduce, motivate, and elaborate on input and output code representations specific to the program repair task.

1) Representation of Fault Localization: Virtually all the APR literature assumes line-based fault localization, with a single line given as input to the repair algorithm. This is not appropriate to fix multi-location bugs [26], [31]. Consider Fig. 3 (OR4), which shows the canonical patch for the multi-location bug Chart-5 from Defects4J. In this case, fault localization must identify a location where an entirely new if block should be synthesized and inserted as well as another pre-existing if condition, appearing later in the code. To our knowledge, there is no fault localization technique able to predict tuples of blocks to be repaired together.

In this paper, we propose a novel way to represent fault localization information: our core idea is to represent fault localization not as a single line, but as a region. In RepairLLaMA, we encode fault localization as a span ranging from the beginning of the suspicious region to its end. This encoding is realistic because 1) identifying a buggy method is within reach of existing fault localization methods, and 2) exhaustively listing all suspicious code regions of a buggy method is worst-case $O(n^2)$ in the number of method lines.

2) Input Representation Space: In APR, the design space of the input representation relates to what is shown from the buggy code and to the presence of additional information. For example, fault localization signals can be useful in scoping down where the code should be modified. However, such information might not be seen at the pre-training stage. For the LLM to utilize it, one must represent it in a way that it can learn during fine-tuning. To study the input representation space, we design four input representations tailored to APR (Fig. 2):

IR1: Buggy function This naive representation describes the code in the standard format as it is written, simply as text. Fig. 2 (IR1) shows the buggy function of the multi-location bug Chart-5, a Defects4J bug. The advantage of IR1 is that it is the same representation LLMs observe during pre-training. When using this representation, the main limitation is that the model has no access to fault localization information and, thus, needs to determine where to change the code, which can be considered as implicit anomaly detection.

IR2: Buggy function w/ FL comments This representation adds two comments signaling the start and end of the buggy chunk of code. For example, in Fig. 2 (IR2), the three lines between the start and end of the suspicious region are surrounded by comments signaling the beginning and end of the buggy chunk. By providing fault localization information, the model can scope its changes to the buggy section.

IR3: Buggy function w/ infilling mask This representation uses the infilling scheme some LLMs are trained for during pre-training [29]. The buggy chunk is replaced by the infilling token, which prompts the model to fill it. For example, in Fig. 2 (IR3), the three lines between the start and end of the suspicious region are replaced by the <*FILL_ME*> token. This representation yields shorter inputs and requires less fine-tuning since the

Input Representations

IR1
```
public XYDataItem addOrUpdate(Number x, Number y) {
 if (x == null) {
  throw new IllegalArgumentException("Null 'x' argument.");
 }
 XYDataItem overwritten = null;
 int index = indexOf(x);
 if (index >= 0 && !this.allowDuplicateXValues) {
  ...
 }
 fireSeriesChanged();
 return overwritten;
}
```

IR2
```
public XYDataItem addOrUpdate(Number x, Number y) {
 if (x == null) {
  throw new IllegalArgumentException("Null 'x' argument.");
 }
 // buggy code starts:
 XYDataItem overwritten = null;
 int index = indexOf(x);
 if (index >= 0 && !this.allowDuplicateXValues) {
 // buggy code ends
  ...
 }
 fireSeriesChanged();
 return overwritten;
}
```

IR3
```
public XYDataItem addOrUpdate(Number x, Number y) {
 if (x == null) {
  throw new IllegalArgumentException("Null 'x' argument.");
 }
 <FILL_ME>
  ...
 }
 fireSeriesChanged();
 return overwritten;
}
```

IR4
```
public XYDataItem addOrUpdate(Number x, Number y) {
 if (x == null) {
  throw new IllegalArgumentException("Null 'x' argument.");
 }
 // buggy code
 // XYDataItem overwritten = null;
 // int index = indexOf(x);
 // if (index >= 0 && !this.allowDuplicateXValues) {
 <FILL_ME>
  ...
 }
 fireSeriesChanged();
 return overwritten;
}
```

Fig. 2. Buggy code of the multi-location bug chart-5 represented in our four different input representations.

Output Representations

OR1
```
public XYDataItem addOrUpdate(Number x, Number y) {
 if (x == null) {
  throw new IllegalArgumentException("Null 'x' argument.");
 }
 if (this.allowDuplicateXValues) {
  add(x, y);
  return null;
 }
 XYDataItem overwritten = null;
 int index = indexOf(x);
 if (index >= 0) {
  ...
 }
 fireSeriesChanged();
 return overwritten;
}
```

OR2
```
 if (this.allowDuplicateXValues) {
  add(x, y);
  return null;
 }
 XYDataItem overwritten = null;
 int index = indexOf(x);
 if (index >= 0) {
```

OR3
```
 public XYDataItem addOrUpdate(Number x, Number y) {
  if (x == null) {
   throw new IllegalArgumentException("Null 'x' argument.");
+ }
+ if (this.allowDuplicateXValues) {
+  add(x, y);
+  return null;
  }
  XYDataItem overwritten = null;
  int index = indexOf(x);
- if (index >= 0 && !this.allowDuplicateXValues) {
+ if (index >= 0) {
   XYDataItem existing = (XYDataItem) this.data.get(index);
   try {
    overwritten = (XYDataItem) existing.clone();
```

OR4
```
   throw new IllegalArgumentException("Null 'x' argument.");
+ }
+ if (this.allowDuplicateXValues) {
+  add(x, y);
+  return null;
  }
@@
  int index = indexOf(x);
- if (index >= 0 && !this.allowDuplicateXValues) {
+ if (index >= 0) {
   XYDataItem existing = (XYDataItem) this.data.get(index);
```

Fig. 3. Patch for multi-location bug chart-5 represented in our four different output representations.

infilling objective has been used during pre-training. However, by masking the buggy portion of code, this representation incurs information loss that can be useful to generate a fix.

IR4: Buggy function w/ infilling mask and buggy code This representation combines the buggy code with the infilling scheme. The buggy code is shown in a comment at the end of the prefix portion. For example, in Fig. 2 (IR4), the buggy lines are kept in comments, and the `<FILL_ME>` token is placed immediately afterward. This representation is different from the one learned during pre-training and requires fine-tuning. Code found in the wild would typically not include buggy code as comments, which is considered bad practice. Yet, with fine-tuning, this representation might add valuable information to the infilling scheme.

3) Output Representation Space: Output representations in APR correspond to the representation of the synthesized fixed code. A natural output representation is a diff over the buggy code, aka a patch. As discussed in subsubsection II-D2, fine-tuning is required to adapt an LLM to generate such task-specific outputs. To study the output representation space, we design four output representations tailored to APR (Fig. 3):

OR1: Fixed function The naive output is the full fixed function. It is not a diff. Fig. 3 (OR1) shows the fixed function of the multi-location bug Chart-5. The major drawback of OR1 is that such output may be much larger than the actual code changes for fixing, and LLMs are known to be more effective at generating short sequences over long sequences.

OR2: Fixed chunk In this representation, the output is composed of the fixed chunk of code to replace the buggy chunk of code. The advantage is that the fixed chunk is typically shorter than the full function, i.e. shorter than OR1. For example, in Fig. 3 (OR2), only 6 fixed lines are outputted. OR2 requires an input representation that includes fault localization (i.e. IR2, IR3, IR4) since the output contains no information regarding what to replace.

OR3: Three-line context-diff The output is a typical contextual diff with a three-line context, aka a unified diff. For example, in Fig. 3 (OR3), a unified diff of the statement change is outputted. The main challenge of this representation is that the model needs to learn to locate the bug locations during fine-tuning, which is difficult. Additionally, this representation is also lengthier than generating a fixed chunk (OR2) only.

OR4: One-line context-diff The output is a contextual diff with a shorter, one-line context. OR4 uses a one-line diff

TABLE I
POSSIBLE CODE REPRESENTATION PAIRS FOR FINE-TUNING LLMS FOR AUTOMATED PROGRAM REPAIR. THEY EXPLOIT THE CHARACTERISTICS OF THE APR TASK, INCL. THE PRESENCE OF FAULT LOCALIZATION SIGNALS AND THE NOTION OF "BUGGY CODE"

Code Representations	FL	Aligned w/ PT	Buggy Code
IR1 x OR1	✗	✔/ ✗	✔
IR1 x OR3	✗	✔/ ✗	✔
IR1 x OR4	✗	✔/ ✗	✔
IR2 x OR2	✔	✗/ ✔	✔
IR3 x OR2	✔	✔/ ✔	✗
IR4 x OR2	✔	✗/ ✔	✔

context, making it shorter than OR3. For example, in Fig. 3 (OR4), there are five source code lines less when compared with OR3. Despite this, it is still lengthier than OR2 and also requires the model to learn where to apply the patch.

4) Input/Output Representation Pairs: To utilize an LLM for APR, input and output representations must be carefully paired. This is because all input representations cannot be paired with all output representations. For instance, IR1 cannot pair with OR2 since one cannot apply a fixed chunk to the buggy function without the fault localization information. Table I provides the list of the code representation pairs that are studied in this paper. Each row corresponds to a code representation pair. Column *FL* indicates whether the pair includes or not fault localization information. Column *Aligned w/ PT* provides a relative assessment of the alignment of the representation w.r.t. the pre-training data/objective. A red cross means that the code representation is not aligned with the pre-training data and objective. The left side shows the input and the right the output representations. Column *Buggy Code* indicates whether the pair includes or not the original buggy code.

The first three rows (i.e., IR1xOR1, IR1xOR3, IR1xOR4) include code representation pairs that do not contain fault localization signals. The input is the same across all pairs (IR1), whereas the output can either be the full fixed function (OR1) or a diff (OR3, OR4). The key difference between the pairs is the output length and format.

The latter three rows (i.e., IR2xOR2, IR3xOR2, IR4xOR2) include code representation pairs that contain fault localization information, either as tokens or as infilling, which is specific to program repair. The most aligned representation with pre-training is IR3xOR2 since the pre-trained model has support for infilling. IR2 represents the infilling objective with never-before-seen comments, whereas IR4 keeps the buggy code as comments. The natural output representation to pair with these is OR2 since it only includes the new code to replace the already localized buggy chunk, minimizing output length. Note that we have empirically tested other combinations in a pilot experiment, and the ones not listed in Table I underperform.

E. Choice of Fine-Tuning Dataset

After choosing an initial model and appropriate code representations, the next step is to curate a fine-tuning dataset. First, the dataset must be relevant to the task at hand. In the APR task, a relevant dataset usually includes pairs of buggy and fixed code samples. Second, the type of samples included should be similar to the target bugs. Third, the size of the dataset should be considered. A larger dataset generally leads to better model performance as it provides more examples for the model to fine-tune from. However, it is important to balance size with quality - a smaller, high-quality dataset may be more beneficial than a larger, low-quality one. Fourth, the diversity of the dataset is important. A diverse dataset that covers a wide range of examples can help the model generalize better to unseen data. Lastly, the legality and ethics of the dataset should be considered, in particular regarding privacy and copyright.

F. Program Repair Adapters for LLMs

With the recent release of various LLMs, the scale of parameters has significantly increased. For instance, state-of-the-art models such as LLaMA [19] and CodeLLaMA [24] range from 7B to 70B parameters. Fine-tuning these LLMs often requires substantial GPU resources. As an example, Lv et al. [32] report that fine-tuning the full parameters of LLaMA-7B on an RTX 3090 consumes 126.08 GB at peak GPU memory usage, with the batch size and sequence length set to 1 and 1024 respectively. Fine-tuning current LLMs with limited resources is a challenge.

RepairLLaMA uses LoRA [20], a state-of-the-art parameter-efficient fine-tuning method that reduces memory requirements while maintaining model performance. Instead of fine-tuning all model parameters, LoRA freezes the pre-trained LLM weights and injects trainable low-rank matrices into specific attention layers. In addition to reducing memory requirements, by reducing the number of trainable parameters LoRA also acts as a regularizer [21] helping prevent overfitting during fine-tuning.

In our implementation, we apply LoRA to the query (`q_proj`) and value (`v_proj`) projection matrices in each transformer's self-attention layer. These projection matrices transform the input embeddings into query and value vectors used in the attention. For each target weight matrix $A \in \mathbb{R}^{d \times k}$, LoRA decomposes the update into a product of two lower-rank matrices $B \in \mathbb{R}^{d \times r}$ and $C \in \mathbb{R}^{r \times k}$, where r is the rank. During inference, the effective weight matrix is $W = W_0 + BA$, where W_0 is the frozen pre-trained weight.

The trained matrices can be interpreted as a *repair adapter* that is much smaller than the original LLMs. For example, with rank r = 8, our adapter only requires training 0.39%[1] of the parameters compared to full fine-tuning. This parameter-efficient approach allows RepairLLaMA to achieve strong program repair performance while being trainable on a single GPU.

G. Inference Time

The final step is to deploy the repair adapter. The target buggy program is fed to a fault localization algorithm

[1]Take CodeLLaMA-7B as an example, its hidden size is 4096, so the original parameter of a matrix is 4096 × 4096. As for the LoRA adapter, this value is 4096 × 8 + 4096 × 8.

and processed to generate an APR-specific code representation. Then, the code representation is fed to the initial model combined with the LoRA repair adapter to generate a list of candidate patches for the buggy program. Patches are then checked for plausibility and correctness per off-the-shelf techniques.

III. Experimental Methodology

A. Research Questions

In this work, we focus on the following research questions:

- **RQ1 (Code Representations for Fine-Tuning)**: What is the best code representation to fine-tune an LLM for program repair?
- **RQ2 (Parameter-Efficient Fine-Tuning vs. Full Fine-Tuning)**: How does parameter-efficient fine-tuning compare against full-parameter fine-tuning for program repair?
- **RQ3 (RepairLLaMA vs. ChatGPT-based APR)**: How does RepairLLaMA compare against state-of-the-art ChatGPT-based program repair?

B. Implementation

Model to Fine-Tune Per the criteria of subsection II-C, we choose CodeLlama-7b [24] as our initial LLM. CodeLLaMA is a publicly available LLM released in 2023 and is trained on 500B code tokens. Per the experiments reported in [24], CodeLLaMA outperforms GPT-3.5 on two code generation benchmarks.

Fine-tuning Dataset We choose Megadiff [33] as the fine-tuning dataset, and process all samples into the different code representations. First, the function pairs – each comprising a buggy version and its fixed counterpart – are extracted along with their corresponding diff identifiers. Subsequently, we eliminate pairs that do not change single functions, and remove duplicate pairs through textual comparison. After that, we compute our custom code representations. We keep only samples whose total length (input plus output) is shorter than 1024 tokens measured by the LLM tokenizer. Consequently, the fine-tuning datasets range from 30,000 to 50,000 fine-tuning pairs (see our appendix repository at https://github.com/ASSERT-KTH/repairllama).

Evaluation Benchmark We select three Java benchmarks for our evaluation: Defects4J [22], HumanEval-Java [8], and GitBug-Java [23]. Following recent related work [11], [14], [34], we scope our evaluation to single-function bugs, as defined in subsection II-B. Defects4J comprises 835 real-world bugs from 17 open-source Java projects, from which we identify 488 single-function bugs. HumanEval-Java is a bug benchmark containing artificial bugs inserted in HumanEval [35] Java programs. HumanEval-Java contains 162 single-function bugs. GitBug-Java is a bug benchmark of recent bugs, collected from the 2023 commit history of 55 open-source repositories, comprising 90 single-function bugs. Contrary to Defects4J, GitBug-Java suffers from less data leakage in the pre-training data since all bugs are much more recent than Defects4J. Notably, GitBug-Java exclusively contains bugs from after the training data cutoff date of all models used in our experiments: CodeLlama-7b (September 2022) [24], *gpt-4-0613* (September 2021)[2], and *gpt-3.5-turbo-0613* (September 2021)[2].

Fine-Tuning Hyperparameters We fine-tune CodeLLaMA with LoRA for each of our curated code representations with the same hyper-parameter settings: we set the learning rate to 5e-4 with cosine decay, max input length to 1024[3], training epoch to 2, and batch size to 16 per GPU, and we use Adam_W as the optimizer. For LoRA, we use a rank of 8, alpha of 16, dropout of 0.05, and inject the adaptation matrices in the *q_proj* and *v_proj* layers. Using the same hyper-parameter settings for each code representation ensures fair comparison. Each fine-tuning run is executed on a server with 4xA100 40GB GPUs.

Inference Setup In inference, we employ beam search as our decoding strategy with a beam size of 10 per previous research [8]. Hence, for each bug, we generate 10 candidate patches. We use the HuggingFace transformers library to implement all fine-tuning and inference experiments. Inference is run on a single A100 40GB GPU.

C. Patch Assessment

Patch assessment is a notoriously hard task. In our evaluation, we compute the best metrics for that task [8], [11], [26]: 1) A *plausible patch* is defined as one that successfully passes all test cases. 2) An *exact-match patch* is textually identical to a developer-provided reference patch, incl. spaces and formatting. 3) An *AST-match patch* has an AST which is equivalent to the AST of the developer-provided reference patch. 4) A *semantic-match patch* is a patch deemed equivalent after manual assessment by an expert.

Plausible and exact-match patches are straightforward. Let us dwell on the two other kinds.

The major advantage of an *AST-match* patch is to compute performance regardless of formatting and indentation changes. It is also more scalable than manually checking patches for correctness without expertise in the programs under repair. The *AST-match* process involves converting plausible and reference patches into abstract syntax trees [36] and subsequently utilizing AST differencing [37] to compare their ASTs for discrepancies.

A *semantic-match* patch is the most costly assessment to get. In this paper, to assess semantic equivalence, the two first authors independently label all plausible but not AST/Exact match patches in a first round. For the patches the two first authors disagree upon, the third author breaks the tie.

For all four metrics, the higher the metric, the better the performance. We validate the candidate patches on a workstation with an 18-core Intel Core i9-10980XE CPU and 128 GB of RAM, operating under Ubuntu 22.04.3 LTS.

[2]https://platform.openai.com/docs/models

[3]Although longer input lengths enable the model to process more context and, potentially, fix more bugs, about four times more GPU memory is required when doubling the input length.

D. Methodology for RQ1

The objective of RQ1 is to investigate the most effective code representations for fine-tuning an LLM for program repair. While existing research has delved into the utility of LLMs for program repair, the impact of the code representations, such as their realism, has been overlooked. It is known that variations in code representations may yield substantial differences in performance for fine-tuned LLMs [25]. Consequently, in RQ1, we empirically evaluate 6 realistic code representation pairs presented in II-D and measure their performance.

We fine-tune an LLM as described in III-B. We prompt the model to generate 10 patches for each bug using beam search decoding. We then evaluate the generated patches as outlined in subsection III-C, to measure the effectiveness of each code representation. We prompt the non fine-tuned CodeLLaMA-7B as a baseline.

We assess the statistical significance of performance difference by employing the McNemar test for each pairwise combination of representations. In our context, each statistical test looks at the binary outcomes of two representations (or models) evaluated on the same set of benchmark examples, according to semantical match. The null hypothesis (H_0) for the McNemar's tests is that the distributions are indistinguishable, i.e. the row and column marginal frequencies in the 2x2 contingency table are equal, i.e., $H_0 : P(A = 1, B = 0) = P(A = 0, B = 1)$, where A and B represent whether A and B, respectively, fixed a given bug.

E. Methodology for RQ2

The objective of RQ2 is to evaluate the respective effectiveness of parameter-efficient and full-parameter fine-tuning. Generally, parameter-efficient fine-tuning methods represent a trade-off between computational cost and model performance, in order to train LLMs with limited computational resources. While traditional full-parameter fine-tuning approaches often yield better results, it comes at the expense of significantly higher memory requirements and a larger-scale fine-tuning dataset. In other words, fully fine-tuning an LLM on a small fine-tuning dataset may result in overfitting. In this experiment, we explore and compare the effectiveness of parameter-efficient and full-parameter fine-tuning in the specific context of program repair, which has never been done to the best of our knowledge.

Baseline. We consider two baseline base models in RQ2: CodeLLaMA-7B and deepseek-coder-6.7b-base [38]. To study the difference between parameter-efficient fine-tuning and full-parameter fine-tuning, we fine-tune both models with both approaches. Here, we use the same hyper-parameters as in LoRA fine-tuning described in RQ1. We use a learning rate of 2e-5 for full-parameter-fine-tuning. We employ the same statistical significance test as described in subsection III-D when comparing RepairLlama against these baselines.

Additionally, we benchmark our approach against the highest-performing model documented by Jiang et al. [34]—specifically, the fine-tuned variant of Incoder-6B [39]. We also compare with RAP-Gen [10], a state-of-the-art model for automated program repair that enhances processing of buggy inputs by leveraging fix patterns retrieved from an extensive repository of historical bug-fix pairs.

```
There's a bug in the Java program below. Try to fix it and return the
complete fix for the code in the form of the markdown code block.
Generate the code to replace the <FILL_ME> token.

[Buggy function represented with IR4xOR2]
```

Fig. 4. The prompt used to prompt GPT-3.5 and GPT-4 as a strong baseline to generate patches.

F. Methodology for RQ3

Recently, related work [14], [40] has shown that GPT-3.5-Turbo and GPT-4 achieve state-of-the-art results on program repair. The objective of RQ3 is to study how RepairLLaMA compares against state-of-the-art ChatGPT-based program repair.

First, we zero-shot prompt *gpt-3.5-turbo-0613* and *gpt-4-0613* to generate 10 patches for each bug. The prompt is shown in Fig. 4, it is built by integrating the best code representation with an effective zero-shot prompt from Zhang et al. [40]. The prompt instructs the LLM to generate the fixed code chunk to replace the <FILL_ME> token. Note that the same information signals are provided to both RepairLLaMA models and the GPT models, allowing for a fair comparison of the repair capabilities of the different models. This includes providing the same source code context, as well as the same fault localization information as input. We employ the same statistical significance test as described in subsection III-D. We utilize OpenAI's official APIs to call *gpt-3.5-turbo-0613* and *gpt-4-0613* to conduct our experiments on Dec. 1, 2023, except for the GitBug-Java experiments which are conducted on *gpt-3.5-turbo-0125* on Nov. 17, 2024.

IV. Experimental Results

A. Results of RQ1 (Code Representations for Fine-Tuning)

In RQ1, we investigate the most effective code representations for fine-tuning an LLM for program repair. The results of the evaluation are presented in Table II, which shows the effectiveness of each code representation setting on both test benchmarks. The table is structured as follows: the first column displays the code representations, the second to fourth meta-columns show the repair effectiveness results on, respectively, the single-function subsets of Defects4J v2, HumanEval-Java, and GitBug-Java. Recall that the repair effectiveness evaluation is measured by the four patch assessment metrics described in subsection III-C.

Our baselines (rows 1 and 2) use the original non-fine-tuned CodeLLaMA-7B, with the IR3xOR2 and IR4xOR2 code representations respectively. We computed the baseline performance for other representations, and these are the most effective way to prompt the non-fine-tuned model [8], making them the strongest possible baselines.

Our results show that the IR3xOR2 baseline plausibly repairs 131 Defects4J v2, 107 HumanEval-Java bugs, and 17 GitBug-Java bugs. Moreover, it correctly repairs 52 Defects4J v2, 71

TABLE II
REPAIR RESULTS OF DIFFERENT CODE REPRESENTATIONS FOR FINE-TUNING AN LLM FOR PROGRAM REPAIR (RQ1). OUR BEST MODEL, REPAIRLLAMA USING IR4XOR2, SIGNIFICANTLY IMPROVES OVER THE BASELINE IN ALL TEST BENCHMARKS

	Code Representations	**Defects4J v2 (488 bugs)**				**HumanEval-Java (162 bugs)**				**GitBug-Java (90 bugs)**			
		Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match
1	IR3 x OR2 (no fine-tuning)	131	52	70	83	107	71	81	103	17	8	9	12
2	IR4 x OR2 (no fine-tuning)	107	50	60	69	95	65	72	91	19	11	12	13
3	IR1 x OR1	79	29	31	45	78	52	54	72	10	4	4	4
4	IR1 x OR3	41	15	17	24	39	21	21	37	6	1	1	1
5	IR1 x OR4	12	2	2	3	5	2	2	4	1	1	1	1
6	IR2 x OR2	**198**	121	122	139	**118**	69	77	108	23	**16**	**17**	19
7	IR3 x OR2	153	83	86	102	103	63	68	99	21	12	12	13
8	**IR4 x OR2 (RepairLLaMA)**	195	**124**	**125**	**144**	**118**	**75**	**82**	**109**	**25**	15	16	**20**

HumanEval-Java, and 8 GitBug-Java bugs with patches that textually exactly match the developer-written ones. Furthermore, when considering AST match, it can repair 70 Defects4J v2, 81 HumanEval-Java, and 9 GitBug-Java bugs with patches that are syntactically equal to the developer-written ones. Finally, when considering semantic match, the baseline can repair 83 Defects4J v2, 103 HumanEval-Java, and 12 GitBug-Java bugs with patches that are semantically equivalent to the developer-written ones. Those results show that CodeLLaMA-7B with the appropriate prompting is already a very strong model for program repair.

The difference between the first baseline (row 1, IR3xOR2) and the second baseline (row 2, IR4xOR2) is that IR4 includes the buggy code as a comment. Despite this extra signal, the baseline model cannot make use of it effectively. Indeed, it achieves worse performance, correctly repairing 14 (69 vs. 83 of IR3xOR2) less Defects4J bugs, 12 (91 vs. 103) less HumanEval-Java bugs, and 1 (13 vs. 12) more GitBug-Java bug. This finding clearly shows that without any fine-tuning, inserting extra information into the representation such as the buggy code can act as noise.

The fine-tuned model's effectiveness depends on the code representations. First, we observe that the three code representation pairs that do not have access to fault localization (IR1xORX, rows 3 to 5) perform considerably worse than both the baseline and other code representations. These results show that fault localization signals are crucial for program repair. To that extent, all representations that simply use the full function as the input and ask to "fix the bug", can be considered too naive. Our work demonstrates that tailoring code representations with fault localization is a necessary step in the context of program repair. Recall that no pre-training objective has access to fault-localization signals, which means that fine-tuning is essential to build knowledge of fault-localization signals.

Second, we observe that, within code representations that use fault localization signals (IR2, IR3, and IR4, rows 6 to 8), fine-tuned models significantly outperform the baseline on Defects4J v2 and GitBug-Java compared to the baselines (rows 1 and 2). Again, this clearly demonstrates the importance of fault localization signals in the input representation.

The only exception to this is in HumanEval-Java, where the fine-tuned IR3xOR2 model performs worse than the baseline IR3xOR2 model. We hypothesize that this phenomenon occurs due to catastrophic forgetting: fine-tuning improves the model for a downstream task at the expense of some loss of knowledge from previous training. In our case, the catastrophic forgetting of HumanEval-Java is explained by the difference in nature between the fine-tuning dataset and the benchmark. Megadiff, the fine-tuning dataset, contains diffs sampled from Java projects hosted on GitHub. The nature of such projects is closer to the nature of Defects4J and GitBug-Java samples, as these are also collected from open-source Java repositories. HumanEval-Java, on the other hand, is manually engineered on top of simple coding tasks not commonly found on large open-source projects, and thus has a different distribution of bug and fix kinds. These results highlight the need for careful curation of fine-tuning datasets and benchmarks. Additionally, it also supports the effectiveness of IR4xOR2, which is able, to a certain degree, to alleviate catastrophic forgetting.

The best model, RepairLLaMA, fine-tuned with the code representation IR4xOR2, plausibly repairs 195 (+64) Defects4J bugs than the baseline, exactly repairs 124 (+72), syntactically correctly repairs 125 bugs (+55), and semantically correctly repairs 144 bugs (+61) bugs, respectively. This highest performance is due to the presence of the buggy code in the input, which gives the model unique ingredients for understanding the problem and generating the patch.

To strengthen the external validity of our analysis beyond Defects4J, we perform the same experiment on HumanEval-Java and GitBug-Java. On HumanEval-Java, RepairLLaMA also achieves better repair effectiveness than the baseline models (75 vs. 71 exact match, 82 vs. 81 AST match, and 109 vs. 103 semantic match). On GitBug-Java, RepairLLaMA also beats the baseline models (15 vs. 11 exact match, 16 vs. 12 AST match, and 20 vs. 13 semantic match). These results are consistent with the observations made on Defects4J.

Now, we discuss the statistical significance of our results. All exact p-values are provided as supplemental material [41]. We observe that there is a statistically significant difference between the golden model RepairLLaMA (IR4xOR2) and all

```
@@ -111,9 +111,6 @@
  final double[] lI = lTData[i];
- if (lTData[i][i] < absolutePositivityThreshold) {
-  throw new NotPositiveDefiniteMatrixException();
- }
  for (int j = i + 1; j < order; ++j) {
   final double[] lJ = lTData[j];
@@ -134,6 +131,9 @@
  final double[] ltI = lTData[i];
+ if (ltI[i] < absolutePositivityThreshold) {
+  throw new NotPositiveDefiniteMatrixException();
+ }
  ltI[i] = Math.sqrt(ltI[i]);
  final double inverse = 1.0 / ltI[i];
```

Fig. 5. Exact match patch generated by RepairLLaMA for math-86 from Defects4J v2. In this multi-location bug, RepairLLaMA is able to fix two distant buggy locations.

```
@@ -6,16 +6,18 @@
    int CAP = 0;
    int SM = 0;
    for (char c : s.toCharArray()) {
-    if (Character.toUpperCase(c) == c) SM += 1;
-    if (Character.toLowerCase(c) == c) CAP += 1;
+    if (Character.toUpperCase(c) == c) CAP += 1;
+    if (Character.toLowerCase(c) == c) SM += 1;
    }
-   val = CAP - SM;
-   strong = s;
+   if (CAP - SM > val) {
+    val = CAP - SM;
+    strong = s;
+   }
   }
  return class_name + "." + strong;
 }
```

Fig. 6. Exact match patch generated by RepairLLaMA for *STRONGEST_EXTENSION* from HumanEval-Java. In this multi-location bug, RepairLLaMA modifies two if blocks and encapsulates two other statements in a new if block.

other models, with very low p-values ($p \leq 1.27e-04$), with the exception of IR2xOR2 for which the test fails to reject the null hypothesis, this case is further discussed below. These results completely validate the importance of the task-specific signals embedded in our best-performing code representations.

Third, we discuss the impact of fine-tuning in aligning the repair-specific code representation and the pre-training objectives. IR2 and IR4 are input representations that include the same signals: the original function, fault localization, and the original buggy code. The key difference is that IR4 uses the same infilling scheme seen during pre-training, while providing the original buggy code in comments, while IR2 places two comments as delimiters of the buggy region. We see a marginal performance gap between both representations (e.g., 144 vs. 139 correctly fixed Defects4J bugs) that is not statistically significant ($p \approx 0.545$) [41]. We hypothesize that, while IR2 does not implicitly use the infilling scheme, fine-tuning effectively maps the delimiting comments of IR2 to the <FILL_ME> token used by IR4 and the pre-trained infilling scheme.

In addition to comparing performance, we explore the effectiveness of region-based representations in addressing multi-location bugs. Our findings reveal that RepairLLaMA correctly repairs 51 instances of such bugs. For example, RepairLLaMA correctly fixes a complex multi-chunk bug, Math-86, from the Defects4J v2, as illustrated in Fig. 5. Math-86 presents two error sections that require simultaneous attention and correction: 1) the removal of an if block that throws an exception, and 2) the introduction of a new if condition. Note that these two sections have more than 20 lines of distance between each other, showing that RepairLLaMA can fix bugs where the multiple edit locations are far away from each other. To summarize, our experiments give clear evidence that RepairLLaMA's representation enables the repair of a wide range of multi-location bugs.

Fig. 6 shows another example of an exact match multi-location patch generated by RepairLLaMA, which is for a HumanEval-Java bug *STRONGEST_EXTENSION*. RepairLLaMA first swaps the statements conditioned by the two existing if conditions, understanding that the wrong counters are being incremented in each case. Then, RepairLLaMA conditions the two statements updating the current values of *val* and *strong* only if the difference between the counters is greater than the already existing solution. Both Figs. 6 and 5 show the effectiveness of RepairLLaMA in repairing multi-location bugs, thanks to tailored code representations that represent fault localization information in a realistic manner.

Answer to RQ1: Our results demonstrate the importance of designing specific code representations for fine-tuning LLMs for APR. Naive representations such as full function are suboptimal, whether on the input or the output side of the model. Our experiments show that the best code representation pair is IR4xOR2, which is due to leveraging two essential signals specific to the program repair task at hand (fault localization and the original buggy code). The RepairLLaMA model, fine-tuned with IR4xOR2, correctly repairs 144 Defects4J bugs, 109 HumanEval-Java bugs, and 20 GitBug-Java bugs, which is state of the art. This significant performance demonstrates the need for curated code representations in automated program repair. While the community focuses a lot on prompt engineering, our original experimental results encourage alternative research on domain-specific, expert code representations per downstream task in software engineering.

B. Results of RQ2 (Parameter-Efficient Fine-Tuning vs. Full Fine-Tuning)

In RQ2, we study how parameter-efficient fine-tuning compares against basic full-parameter fine-tuning. Recall most of the closely related work in APR [5], [8], [9], [10], [12], [42] use full-parameter fine-tuning. On the contrary, the key novelty of RepairLLaMA is that all models are fine-tuned using LoRA, a parameter-efficient fine-tuning technique that optimizes only a small adapter (approx. 4M parameters) instead of the whole

TABLE III
REPAIR EFFECTIVENESS OF PARAMETER-EFFICIENT FINE-TUNED MODELS COMPARED WITH FULLY FINE-TUNED MODELS (RQ2). REPAIRLLAMA, TRAINED WITH PARAMETER-EFFICIENT FINE-TUNING, OUTPERFORMS ITS VARIANTS ON DEFECTS4J, HUMANEVAL-JAVA, AND GITBUG-JAVA

Base Model	**Fine-Tuning Method**	**Defects4J v2 (488 bugs)**				**HumanEval-Java (162 bugs)**				**GitBug-Java (90 bugs)**			
		Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match
codellama-7b	-	131	52	70	83	107	71	81	103	17	8	9	12
codellama-7b	FFT	146	66	84	98	109	74	**83**	100	21	11	11	13
codellama-7b	LoRA	**195**	**124**	**125**	**144**	**118**	**75**	82	**109**	**25**	**15**	**16**	**20**
deepseek-coder-6.7b	Base	147	84	88	104	113	68	83	110	21	11	11	13
deepseek-coder-6.7b	FFT	**185**	**116**	**123**	**138**	129	96	101	119	**26**	**15**	**15**	**19**
deepseek-coder-6.7b	LoRA	181	110	113	128	**134**	**99**	**107**	**124**	19	12	12	13

LLM (approx. 7B parameters, a reduction of approx. 1600x). This allows the model to 1) be fine-tuned with less GPU memory, and 2) potentially reduce overfitting. In RQ2, we compare RepairLLaMA, built with the best code representation in RQ1, with its full-parameter fine-tuning version. Additionally, we also study the same fine-tuning methods with a different base model, *deepseek-coder-6.7b*, to increase external validity.

Table III presents the results of RQ2. The table reads as follows. The first column presents the base model. The second column presents the fine-tuning model. The third to fifth meta-columns show the repair effectiveness results on, respectively, Defects4J v2, HumanEval-Java, and GitBug-Java.

The results show that RepairLLaMA with parameter-efficient fine-tuning clearly outperforms both the baseline and its full-parameter fine-tuning version. In Defects4J, RepairLLaMA plausibly repairs 49 (195 vs. 146) bugs more and correctly repairs, according to semantic match, 46 (144 vs. 98) more than the fully fine-tuned model. When considering AST Match, RepairLLaMA also repairs 41 (125 vs. 84) more bugs. In HumanEval-Java and GitBug-Java, although the improvement is smaller, RepairLLaMA still outperforms all full fine-tuning baselines. We find a statistical significance between the golden model IR4xOR2 and the two baselines ($p \leq 1.29e-03$, see [41]).

The gain in performance is clear, and it is actually a double win because RepairLLaMA requires fewer resources. Possibly, the fully fine-tuned models suffer from overfitting due to the limited fine-tuning data, while parameter-efficient fine-tuning helps prevent overfitting since it only requires optimizing a small part of the network weights. The model size constraints of LoRA appear to act as implicit regularizers in this experiment. These results confirm the findings of Fu et al. [21]: parameter-efficient fine-tuning helps mitigate overfitting by leveraging trainable parameter sparsity and reducing the generalization error bound. Yet, a balance is needed to prevent underfitting due to insufficient trainable parameters.

This phenomenon is not observed for *deepseek-coder-6.7b*. For this model, fine-tuning clearly improves over the base model. The best fine-tuned model, for each benchmark, repairs 34 (138 vs. 104) more Defects4J, 24 (107 vs. 83) more HumanEval-Java, and 6 (19 vs. 13) more GitBug-Java bugs than the base model. The full fine-tuned version outperforms LoRA in Defects4J and GitBug-Java, while the LoRA version outperforms in HumanEval-Java. There is a statistically significant difference between both the fully fine-tuned ($p \approx 2.72e-05$) and LoRA versions ($p \approx 4.21e-04$) of deepseek-coder-6.7b when compared with the baseline model, meaning that both methods perform a statistically significant improvement over the baseline model. These results suggest that the effect of parameter-efficient fine-tuning varies across models. It can under- or over-perform depending on the model and benchmark, always with the advantage of smaller resource requirements.

When compared with the best fully fine-tuned model from Jiang et al. [8], which keeps the LLMs' original pre-training format but includes the buggy lines as comments, RepairLLaMA correctly repairs 25 (95 vs. 69) Defects4J bugs more, when applied to the same dataset considered in Jiang et al. and 39 (109 vs. 70) HumanEval-Java bugs more. In addition, RepairLLaMA correctly repairs 49 more Defects4J multi-hunk bugs. Notably, RepairLLaMA achieves these results without access to perfect bug localization information, which is assumed and incorporated in [8].

To further evaluate RepairLLaMA, we also compare it with RAP-Gen [10], the state-of-the-art program repair approach achieved by full-parameter fine-tuning CodeT5. RAP-Gen builds code representations by augmenting the original input of buggy code with retrieved bug-fix pairs. RAP-Gen correctly repairs 125 Defects4J v2 bugs, according to the authors' own manual verification, when generating 100 patches per bug and providing perfect line-level fault localization. In contrast, RepairLLaMA correctly repairs 19 more Defects4J v2 bugs (144 vs. 125), while generating only 10 patches per bug (10x less than RAP-Gen) and operating under more realistic fault localization. This clearly demonstrates that RepairLLaMA improves the state-of-the-art of fine-tuning for program repair.

Fig. 5 shows a multi-location bug from Math-86 (Defects4J v2) where RepairLLaMA successfully generates an exact match patch by fixing two distant buggy locations simultaneously. This example clearly demonstrates the limitations of both comparison approaches: Jiang et al.'s approach can only handle single-chunk bugs, and its representation is inadequate for addressing bugs that span multiple locations, as shown in this example. RAG-Gen, while theoretically capable, requires perfect fault localization information, which is impractical in

TABLE IV
REPAIRLLAMA'S EFFECTIVENESS COMPARED WITH STATE-OF-THE-ART CHATGPT-BASED APR TECHNIQUES (RQ3). REPAIRLLAMA IS MORE EFFECTIVE IN FINDING CORRECT AND PLAUSIBLE PATCHES IN DEFECTS4J AND GITBUG-JAVA, INCL. AGAINST THE STRONG BASELINE OF GPT-4

Model	**Defects4J v2 (488 bugs)**				**HumanEval-Java (162 bugs)**				**GitBug-Java (90 bugs)**			
	Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match	Plausible	Exact Match	AST Match	Semantic Match
GPT-3.5	71	23	33	45	107	50	63	97	9	7	7	8
GPT-4	119	47	60	72	**124**	64	74	**116**	14	6	7	10
RepairLLaMA (IR4xOR2)	**195**	**124**	**125**	**144**	118	**75**	**82**	109	**25**	**15**	**16**	**20**

real-world scenarios. In contrast, RepairLLaMA overcomes these limitations by tailoring effective code representation that can handle complex multi-location bugs without requiring perfect fault localization information.

Answer to RQ2: The efficacy of fine-tuning is clear, yet depends on the base model. RepairLLaMA's parameter-efficient fine-tuning outperforms full fine-tuning of *codellama-7b*, with external validity over three benchmarks, while results for *deepseek-coder-6.7b* show similar improvements regardless of the fine-tuning method. Overall, our paper is the first to demonstrate the benefits of parameter-efficient fine-tuning in the context of automated program repair, achieving strong state-of-the-art results. Beyond program repair, we note that parameter-efficient fine-tuning is feasible within academic labs' budget and hardware, while still working with powerful multi-million dollar trained large models.

C. Results of RQ3 (RepairLLaMA vs. ChatGPT-Based APR)

In RQ3, we compare RepairLLaMA with state-of-the-art zero-shot ChatGPT-based program repair. ChatGPT-based program repair differs from RepairLLaMA since it does not involve fine-tuning LLMs on task-specific datasets. Instead, it involves designing effective prompt strategies to instruct a powerful general-purpose LLM like GPT-4.

Table IV shows the repair effectiveness of RepairLLaMA compared with ChatGPT-based APR techniques. The first column indicates the model name. The second, and third show the results on two different benchmarks. Each benchmark is evaluated following the four patch assessment metrics described in subsection III-C.

Our results show that RepairLLaMA is the most effective model. GPT-3.5 and GPT-4 plausibly fix 64% and 39% less Defects4J v2 bugs than RepairLLaMA, respectively. RepairLLaMA correctly fixes twice as many bugs as GPT4, when considering any of the correct fix metrics (i.e. Exact/AST/Semantic Match). GPT-4 correctly repairs more HumanEval-Java bugs than RepairLLaMA according to semantic match, which may be due to overfitting to HumanEval. RepairLLaMA correctly repairs more according to both Exact/AST Match metrics.

RepairLLaMA also plausibly and correctly fixes more GitBug-Java bugs than both GPT-3.5 and GPT-4, showing external validity. GitBug-Java exclusively contains bugs from 2023, meaning that all bugs occurred after the training data cutoff date of all three models. Moreover, we observe a statistically significant difference between RepairLLaMA and GPT-3.5/GPT-4 ($p \leq 4.10e-08$, see [41]). Recall that both GPT-3.5 and GPT-4 are larger than the RepairLLaMA model by several orders of magnitude, making RepairLLaMA's specialized performance remarkable.

To conclude, our experimental results demonstrate the power of specializing an LLM for APR over zero-shot learning of a foundational model. A smaller model, trained with a parameter-efficient fine-tuning technique, is more effective than a large general-purpose LLM. Overall, RepairLLaMA beats the strong baseline of GPT-4 on the hardest benchmark Defects4J.

Answer to RQ3: RepairLLaMA beats GPT-4, thanks to the combination of appropriate code representations and parameter-efficient fine-tuning. Our experiments demonstrate the need for specialized representations and specialized training for program repair. The RepairLLaMA program repair adapter is more powerful than ChatGPT for fixing bugs in Java.

V. DISCUSSION

A. Sampling Candidate Patches

One key aspect of any program repair approach is the number of generated candidate patches. Some recent works generate hundreds and even thousands of patches for a single bug [7], [10], [14], [43], [44]. However, the cost of evaluating such a large number of candidate patches has been largely overlooked. Recall that to evaluate the plausibility of each candidate patch, one must run the test cases, which is expensive and even overcomes the one-time cost of fine-tuning. In contrast to this trend, RepairLLaMA achieves state-of-the-art results while generating only 10 candidate patches per bug. This shows that 1) RepairLLaMA natively priorities the best patches in the top-10 list, 2) RepairLLaMA minimizes the resources that are required in an end-to-end repair pipeline that includes plausibility checking.

B. Fault Localization With Code Regions

In most recent program repair articles, effectiveness measurement assumes perfect fault localization (FL). Such an assumption, while convenient for research purposes, is unrealistic in real-world scenarios since current FL techniques cannot reliably locate all faulty code lines with high accuracy. This is particularly true in cases where the buggy lines are spread across different locations since there are virtually no multi-location fault localization algorithms. To enhance practical applicability, we propose in this paper an alternative, relaxed assumption: instead of making a perfect localization assumption, we only assume that one is able to identify a code region that encompasses all the buggy lines.

For a function with N code lines, there are maximum $\frac{N(N+1)}{2}$ code regions. In other words, the faulty code region (i.e., the shortest region containing all buggy lines) can be identified with $O(N^2)$ complexity. This makes it computationally feasible to generate candidate regions by examining the different pairs of start and end lines, if N is not too lage. We implement[4] and analyze[5] the proposed approach for all three considered benchmarks (Defects4Jv2, HumanEval-Java, GitBug-Java). We observe that the 75th percentile of the number of code regions in the buggy functions of each benchmark is approx. 8485 for GitBug-Java, 861 for Defects4J, and 91 for HumanEval-Java. This shows the feasibility of enumerating all possible regions for most single-function bugs of the considered benchmarks.

We believe that this novel strategy is realistically better: even if the localization is imperfect, as long as it can narrow down the search space to a region containing all buggy lines, a model such as RepairLLaMA would be effective in generating multi-location patches.

To sum up, our code region strategy 1) dramatically relaxes the assumption of perfect multi-line fault localization of related research, and 2) enables mor practical program repair.

C. Fine-Tuning vs. Prompt Engineering

Fine-tuning and prompt engineering are two distinct strategies for utilizing LLMs in program repair. Prompt engineering leverages the inherent capabilities of LLMs to generate bug fixes by steering the models with carefully crafted prompts, often including examples and detailed instructions. Prompt engineering requires no additional training and can be rapidly used for new tasks. However, its reliance on ad hoc prompt engineering introduces variability (e.g., what works for one model might not work for another), bounds the amount of task-specific information that can be provided to the model to its context window, and is usually dependent on large, proprietary models.

Fine-tuning, as we do in this paper, adapts the LLM by learning domain-specific tokens, structures and signals. In our program repair context, this means learning the relationship between buggy and fixed code by directly changing the model's weights. This results in consistent, high-quality fixes, even with smaller models. Although fine-tuning requires task-specific datasets and more computation resources, methods like LoRA help mitigate these costs. While both approaches have advantages, our results confirm the effectiveness of fine-tuning in achieving good performance in program repair.

D. Threats to Validity

The primary internal threat lies in the potential data leakage during the pre-training phase of LLMs. LLMs are pre-trained on vast corpora scrapped from the web, potentially including the same data used for testing, endangering the reliability of experimental results. To mitigate this threat, we assess all models on two recent benchmarks specifically designed to address the data leakage issue [45], HumanEval-Java [8] and GitBug-Java [23].

Another internal threat pertains to data leakage during the fine-tuning process of LLMs, since both our fine-tuning dataset, Megadiff, and Defects4J contain samples from GitHub from the same period of time. To address this threat, we meticulously compare the samples in our fine-tuning dataset, Megadiff, with those in Defects4J. We found no identical samples shared by both datasets. However, it is worth noting that there are three samples (Math-28, Math-44, and JacksonDatabind-82) whose patch includes a function also found in Megadiff samples. To mitigate this threat, we exclude these three Defects4J samples from the evaluation of our fine-tuned models. Moreover, both HumanEval-Java and GitBug-Java have been constructed after Megadiff and, as such, are non-overlapping.

The main external threats are two-fold: 1) the focus on a single programming language, as our results may not generalize to other languages, and 2) the focus on a single LLM.

To mitigate the first threat, we evaluate on three benchmarks, including well-established Defects4J [22]. Our core novelties are programming language agnostic and should generalize to arbitrary languages.

Regarding the second threat, we argue that our approach is model agnostic. The only model-specific feature we rely on is the infilling (aka fill-in-the-middle) feature. This feature is commonly used in LLMs' training for code [28], [39], [46], [47], and, as such, our approach is easily extensible to these models. For the LLMs that are not trained with this objective, we note that it is also possible to fine-tune them with the proposed representations.

VI. Related Work

A. Large Language Models for Program Repair

Fine-Tuning. Several works [5], [8], [9], [10], [12], [42], [48], [49], [50], [51], [52], [53] have proposed fine-tuning large language models for the program repair task. Notably, Jiang et al. [8] specifically study the impact of fine-tuning LLMs for program repair, reporting improvements lower than ours while using naive full-parameter fine-tuning. Huang et al. [25] also study different aspects of fine-tuning LLMs for program repair, including code representations and evaluation metrics. While

[4]https://github.com/ASSERT-KTH/repairllama/blob/main/src/fl/fl.py

[5]https://github.com/ASSERT-KTH/repairllama/blob/main/src/fl/analyse_fl.ipynb

they report state-of-the-art performance, it is achieved under the unrealistic assumption of perfect multi-line fault localization, which RepairLLaMA does not assume. Yang et al. [54] propose MORepair, an approach to fine-tuning CodeLlama for program repair with multiple objectives, including LLM-generated guidance text in the novel fine-tuning objective. MORepair takes inspiration from RepairLLaMA in using parameter-efficient fine-tuning to train a repair adapter, comparing MORepair with RepairLLaMA on two new benchmarks built from HumanEval [35].

Overall, our work distinguishes itself from related work in three key aspects. First, we have designed and evaluated several code representations in RepairLLaMA, tailored to fine-tuning LLMs for program repair, which incorporate fault localization signals under realistic assumptions. This is different from previous work (e.g., [10], [25]) which assumes perfect multi-line fault localization. Second, RepairLLaMA is the first to employ LoRA to fine-tune LLMs for program repair, demonstrating that parameter-efficient fine-tuning can surpass full-parameter fine-tuning while reducing computational requirements. Third, unlike some previous work that generates hundreds, even thousands of patches for each bug, our best model, RepairLLaMA, improves state-of-the-art performance on Defects4J and HumanEval-Java with a budget of just 10 patches per bug, demonstrating the laser-style focus of the trained program repair adapter.

Prompting and Agents. Many works [7], [11], [40], [43], [44], [55], [56], [57] directly use LLMs for program repair without any fine-tuning. The core of these works is prompting: they design and evaluate different prompting strategies to provide repair information to the model. Notably, SRepair [44] uses a dual-LLM approach that first generates natural language repair suggestions by providing the buggy function, failing tests, and error messages in a chain-of-thought prompt to an LLM. Then, SRepair prompts a smaller LLM to generate up to 500 patches per bug based on the high-level suggestions. Recently, related work [14], [15], [16], [17] has also proposed agents for program repair. The core novelty is to prompt LLMs iteratively, with intermediate steps for tool execution and incorporation of their feedback in the prompting loop.

Our work is fundamentally different from this related work: 1) we do not do prompt engineering, and 2) we do not iterative repair. RepairLLaMA crystallizes all the repair knowledge in the model weights obtained by parameter-efficient fine-tuning. The concept of program repair adapter is entirely different and new compared to prompting. Remarkably, RepairLLaMa achieves strong performance while being zero-shot without access to external tools and feedback. We consider our findings to be the foundation of future work in fine-tuning for software agents.

B. Code Representations for Program Repair

Several code representations for program repair have been proposed by related work [6], [18], [30], [58], [59], [60], [61], [62], [63]. Notably, Namavar et al. [30] investigate the impact of different code representations for program repair for a restricted set of bug classes. Differently, our work targets a larger spectrum of bugs, including multi-location bugs.

Overall, our work distinguishes itself from preceding research in code representation in three dimensions. First, we design code representations that are aligned with the pre-training data and objectives, enabling the RepairLLaMA to well utilize the pre-learned knowledge. Second, our code representations are designed to support a large spectrum of bugs, including multi-location bugs, which is one frontier of program repair. Third, RepairLLaMA's pipeline and evaluation are not constructed under the unrealistic perfect fault localization for multi-location bugs.

C. Parameter-Efficient Fine-Tuning in SE

Parameter-efficient fine-tuning is a relatively under-explored area in Software Engineering. Wang et al. [64] explore parameter-efficient fine-tuning techniques for specializing LLMs for code search and code summarization, finding that parameter-efficient fine-tuning outperforms in-context learning. Weyssow et al. [65] confirm the dominance of parameter-efficient fine-tuning techniques over zero-shot learning in code generation, while Wang et al. [66] find that prompt-tuning outperforms traditional fine-tuning methods in code summarization. CodePrompt [67] proposes corpus-specific prompt templates similar to adaptations and boosts code generation performance. Shi et al. [68] propose a parameter-efficient fine-tuning technique for code related tasks that selectively freezes layers of the model. Zou et al. [69] study the effectiveness of five parameter-efficient fine-tuning methods in four software engineering tasks, finding them competitive with full-parameter fine-tuning. Lastly, Liu et al. [70] study the performance of Adapter Tuning and LoRA in different software engineering tasks, including cross-lingual and low-resource scenarios, finding PEFT to outperform or achieve comparable results when compared with full-parameter fine-tuning.

Overall, we are the first to employ and evaluate LoRA to fine-tune LLMs for program repair. RepairLLaMA's effectiveness calls for more work in parameter-efficient fine-tuning in program repair and related tasks such as overfitting detection.

VII. Conclusion

In this paper, we have proposed RepairLLaMA, a novel program repair approach that combines parameter-efficient fine-tuning with program repair-specific code representations. RepairLLaMA's code representations are unique in incorporating repair signals, such as fault localization, under realistic assumptions, and in aligning with pre-training data and objectives. To validate RepairLLaMA, we perform a series of extensive experiments on three benchmarks: Defects4J, HumanEval-Java, and GitBug-Java. Our results clearly validate our core design decisions, with RepairLLaMA correctly fixing 144 Defects4J, 109 HumanEval-Java bugs, and 20 GitBug-Java bugs, outperforming strong baselines, incl. GPT-3.5 and GPT-4. RepairLLaMA opens an avenue for research on different kinds of efficient fine-tuning for program repair.

ACKNOWLEDGMENT

The computations/data handling were enabled by the supercomputing resources Berzelius-2023-175 and Berzelius-2024-336 provided by National Supercomputer Centre at Linköping University and the Knut and Alice Wallenberg foundation.

REFERENCES

[1] M. Monperrus, "Automatic software repair: A bibliography," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–24, 2018.

[2] C. Le Goues, M. Pradel, A. Roychoudhury, and S. Chandra, "Automatic program repair," *IEEE Softw.*, vol. 38, no. 4, pp. 22–27, Jul./Aug. 2021.

[3] Z. Chen, S. Kommrusch, M. Tufano, L.-N. Pouchet, D. Poshyvanyk, and M. Monperrus, "SequenceR: Sequence-to-sequence learning for end-to-end program repair," *IEEE Trans. Softw. Eng.*, vol. 47, no. 9, pp. 1943–1959, Sep. 2021.

[4] Q. Zhu et al., "A syntax-guided edit decoder for neural program repair," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, New York, NY, USA: ACM, 2021, pp. 341–353.

[5] N. Jiang, T. Lutellier, and L. Tan, "CURE: Code-aware neural machine translation for automatic program repair," in *Proc. ACM/IEEE 43rd Int. Conf. Softw. Eng.*, 2021, pp. 1161–1173.

[6] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "SelfAPR: Self-supervised program repair with test execution diagnostics," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–13.

[7] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 959–971.

[8] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng. (ICSE),* 2023, pp. 1430–1442.

[9] C. S. Xia, Y. Ding, and L. Zhang, "The plastic surgery hypothesis in the era of large language models," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 522–534.

[10] W. Wang, Y. Wang, S. Joty, and S. C. Hoi, "RAP-Gen: Retrieval-augmented patch generation with codet5 for automatic program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found.s Softw. Eng.*, 2023, pp. 146–158.

[11] C. S. Xia, Y. Wei, and L. Zhang, "Practical program repair in the era of large pre-trained language models," 2022, *arXiv:2210.14179*.

[12] W. Yuan et al., "Circle: Continual repair across programming languages," in *Proc. 31st ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2022, pp. 678–690.

[13] H. Joshi, J. C. Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, "Repair is nearly generation: Multilingual program repair with LLMs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, 2023, pp. 5131–5140.

[14] C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 out of 337 bugs for $ 0.42 each using ChatGPT," 2023, *arXiv:2304.00385*.

[15] J. Yang et al., "Swe-Agent: Agent-computer interfaces enable automated software engineering," 2024, *arXiv:2405.15793*.

[16] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "Autocoderover: Autonomous program improvement," 2024, *arXiv:2404.05427*.

[17] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, LLM-based agent for program repair," 2024, *arXiv:2403.17134*.

[18] Z. Chen et al., "PLUR: A unifying, graph-based view of program learning, understanding, and repair," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 23089–23101.

[19] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning (practical experience report)," 2023, *arXiv:2308.11148*.

[20] E. J. Hu et al., "LORA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2021.

[21] Z. Fu, H. Yang, A. M.-C. So, W. Lam, L. Bing, and N. Collier, "On the effectiveness of parameter-efficient fine-tuning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 11, 2023, pp. 12799–12807.

[22] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *Proc. Int. Symp. Softw. Testing Anal.,* New York, NY, USA: ACM, 2014, pp. 437–440.

[23] A. Silva, N. Saavedra, and M. Monperrus, "Gitbug-Java: A reproducible benchmark of recent Java bugs," in *Proc. IEEE/ACM 21st Int. Conf. Mining Softw. Repositories (MSR)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 118–122.

[24] B. Roziere et al., "Code Llama: Open foundation models for code," 2023, *arXiv:2308.12950*.

[25] K. Huang et al., "An empirical study on fine-tuning large language models of code for automated program repair," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Piscataway, NJ, USA: IEEE Press, 2023, pp. 1162–1174.

[26] H. Ye and M. Monperrus, "ITER: Iterative neural repair for multi-location patches," 2023, *arXiv:2304.12015*.

[27] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[28] R. Li et al., "StarCoder: May the source be with you!," 2023, *arXiv:2305.06161*.

[29] M. Bavarian et al., "Efficient training of language models to fill in the middle," 2022, *arXiv:2207.14255*.

[30] M. Namavar, N. Nashid, and A. Mesbah, "A controlled experiment of different code representations for learning-based program repair," *Empirical Softw. Eng.*, vol. 27, no. 7, p. 190, 2022.

[31] S. Saha, R. K. Saha, and M. R. Prasad, "Harnessing evolution for multi-hunk program repair," in *Proc. 41st Int. Conf. Softw. Eng. (ICSE)*, 2019, pp. 13–24.

[32] K. Lv, Y. Yang, T. Liu, Q. Gao, Q. Guo, and X. Qiu, "Full parameter fine-tuning for large language models with limited resources," 2023, *arXiv:2306.09782*.

[33] M. Monperrus, M. Martinez, H. Ye, F. Madeiral, T. Durieux, and Z. Yu, "Megadiff: A Dataset of 600k java source code changes categorized by diff size," 2021, *arXiv:2108.04631*.

[34] N. Jiang, T. Lutellier, and L. Tan, "Cure: Code-aware neural machine translation for automatic program repair," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng. (ICSE)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1161–1173.

[35] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.

[36] R. Pawlak, M. Monperrus, N. Petitprez, C. Noguera, and L. Seinturier, "SPOON: A library for implementing analyses and transformations of Java source code," *Softw.: Pract. Exp.*, vol. 46, no. 9, pp. 1155–1179, Sep. 2016.

[37] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proc. 29th ACM/IEEE Int. Conf. Automated Softw. Eng.*, 2014, pp. 313–324.

[38] D. Guo et al., "DeepSeek-Coder: When the large language model meets programming–the rise of code intelligence," 2024, *arXiv:2401.14196*.

[39] D. Fried et al., "Incoder: A generative model for code infilling and synthesis," 2022, *arXiv:2204.05999*.

[40] Q. Zhang et al., "A critical review of large language model on software engineering: An example from ChatGPT and automated program repair," 2023, *arXiv:2310.08879*.

[41] "RepairLLaMA-Statistical test," Accessed: Nov. 5, 2024. [Online]. Available: https://github.com/ASSERT-KTH/repairllama/blob/main/src/patch_analysis/statistical_testing.md

[42] A. Shirafuji, M. M. Rahman, M. F. I. Amin, and Y. Watanobe, "Program repair with minimal edits using CodeT5," 2023, *arXiv:2309.14760*.

[43] Y. Wei, C. S. Xia, and L. Zhang, "Copiloting the copilots: Fusing large language models with completion engines for automated program repair," 2023, *arXiv:2309.00608*.

[44] J. Xiang, X. Xu, F. Kong, M. Wu, H. Zhang, and Y. Zhang, "How far can we go with practical function-level program repair?" 2024, *arXiv:2404.12833*.

[45] D. Ramos, C. Mamede, K. Jain, P. Canelas, C. Gamboa, and C. L. Goues, "Are large language models memorizing bug benchmarks?" 2024, *arXiv:2411.13323*.

[46] M. AI, "Codestral: Hello, world!" Accessed: Dec. 4, 2024. [Online]. Available: https://mistral.ai/news/codestral/

[47] C. Team et al., "Codegemma: Open code models based on Gemma," 2024, *arXiv:2406.11409*.

[48] J. Zhang, S. Panthaplackel, P. Nie, J. J. Li, and M. Gligoric, "Coditt5: Pretraining for source code and natural language editing," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–12.

[49] A. Zirak and H. Hemmati, "Improving automated program repair with domain adaptation," *ACM Trans. Softw. Eng. Method.*, vol. 33, no. 3, pp. 1–43, 2024.

[50] M. Jin et al., "InferFix: End-to-end program repair with LLMs," 2023, *arXiv:2303.07263*.

[51] R. Paul, M. M. Hossain, M. L. Siddiq, M. Hasan, A. Iqbal, and J. C. Santos, "Enhancing automated program repair through fine-tuning and prompt engineering," 2023, *arXiv:2304.07840*.

[52] S. Hao, X. Shi, H. Liu, and Y. Shu, "Enhancing code language models for program repair by curricular fine-tuning framework," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME),* Piscataway, NJ, USA: IEEE Press, 2023, pp. 136–146.

[53] S. B. Hossain et al., "A deep dive into large language models for automated bug localization and repair," *Proc. ACM Softw. Eng.,* vol. 1, no. FSE, pp. 1471–1493. 2024.

[54] B. Yang et al., "Multi-objective fine-tuning for enhanced program repair with LLMs," 2024, *arXiv:2404.12636.*

[55] T. Ahmed and P. Devanbu, "Majority rule: Better patching via self-consistency," 2023, *arXiv:2306.00108*.

[56] Q. Huang, J. Zhu, Z. Xing, H. Jin, C. Wang, and X. Xu, "A chain of AI-based solutions for resolving FQNs and fixing syntax errors in partial code," 2023, *arXiv:2306.11981*.

[57] P. Deligiannis, A. Lal, N. Mehrotra, and A. Rastogi, "Fixing rust compilation errors using LLMs," 2023, *arXiv:2308.05177*.

[58] Z. Chen, S. J. Kommrusch, M. Tufano, L. Pouchet, D. Poshyvanyk, and M. Monperrus, "SequenceR: Sequence-to-sequence learning for end-to-end program repair," *IEEE Trans. Softw. Eng.*, 2019, vol. 47, no. 9, pp. 1943–1959, Sep. 2021.

[59] R. Gupta, S. Pal, A. Kanade, and S. Shevade, "DeepFix: Fixing common C language errors by deep learning," in *Proc. 31st AAAI Conf. Artif. Intell. (AAAI)*, AAAI Press, 2017, pp. 1345–1351.

[60] K. Abhinav et al., "RepairNet: Contextual sequence-to-sequence network for automated program repair," in *Proc. Int. Nat. Conf. Artif. Intell. Educ.,* New York, NY, USA: Springer, 2021, pp. 3–15.

[61] Z. Chen, S. Kommrusch, and M. Monperrus, "Neural transfer learning for repairing security vulnerabilities in C code," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, pp. 147–165, Jan. 2023.

[62] M. Yasunaga and P. Liang, "Graph-based, self-supervised program repair from diagnostic feedback," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2020, pp. 10799–10808.

[63] N. Muennighoff et al., "Octopack: Instruction tuning code large language models," 2023, *arXiv:2308.07124*.

[64] D. Wang et al., "One adapter for all programming languages? Adapter tuning for code search and summarization," 2023, *arXiv:2303.15822*.

[65] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring parameter-efficient fine-tuning techniques for code generation with large language models," 2023, *arXiv:2308.10462*.

[66] C. Wang, Y. Yang, C. Gao, Y. Peng, H. Zhang, and M. R. Lyu, "No more fine-tuning? An experimental evaluation of prompt tuning in code intelligence," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 382–394.

[67] Y. Choi and J.-H. Lee, "Codeprompt: Task-agnostic prefix tuning for program and language generation," in *Proc. Findings Assoc. Comput. Linguistics (ACL)*, 2023, pp. 5282–5297.

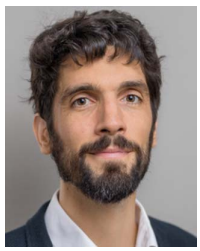
[68] E. Shi et al., "Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond," 2023, *arXiv:2304.05216*.

[69] W. Zou et al., "A comprehensive evaluation of parameter-efficient fine-tuning on software engineering tasks," 2023, *arXiv:2312.15614*.

[70] S. Liu, J. Keung, Z. Yang, F. Liu, Q. Zhou, and Y. Liao, "Delving into parameter-efficient fine-tuning in code change learning: An empirical study," 2024, *arXiv:2402.06247*.

André Silva received the M.Sc. degree in computer science from the Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal. He is currently working toward the Ph.D. degree with KTH Royal Institute of Technology, Stockholm, Sweden. His research interests include the intersection of automatic program repair and machine learning.

Sen Fang is currently working toward the Ph.D. degree with NC State University. He was the Research Engineer with KTH, Royal Institute of Technology, Sweden. His research lies in the intersection between software engineering and machine learning, focusing on AI for software engineering, LLMs for code, and code security.



Martin Monperrus received the master's degree from Compiègne University of Technology and the Ph.D. degree from the University of Rennes. He is a Professor of software technology with KTH Royal Institute of Technology, Sweden. His research lies in the field of software engineering with a current focus on automatic program repair, AI on code and software integrity. For more information, see https://www.monperrus.net/martin/.